

INDU: Fjärilars utbredning i Sverige

Rapport för individuell uppgift i programmering (DA2004/DA2005)

1. Vilken uppgift jag har implementerat

Projektet "Fjärilars utbredning i Sverige" (nivå E-C). Programmet läser observationsdata som exporterats från Artportalen som semikolonseparerade CSV-filer och skapar tre diagram per art: nordligaste observationen per år med referenslinjer för Ystad och Abisko, antalet observationer per år, samt andelen observationer per vecka för ett valt år (2022 som standard). I veckodiagrammet markeras den period då 90 % av årets observationer görs, och samma period skrivs också ut i konsolen. Diagrammen sparas som PDF med filnamn som innehåller art och diagramtyp.

2. Hur programmet startas och används

Programmet startas från projektkatalogen och styrs med flaggor på kommandoraden:

- En art: `python butterfly_analysis.py --csv butterfly_data/amiral.csv --outdir plots`
- Alla filer i en katalog: `python butterfly_analysis.py --csv butterfly_data --all --outdir plots`
- `--species "Amiral"` väljer art i en fil som innehåller flera; utan flaggan analyseras den vanligaste arten i varje fil.
- `--year 2021` byter år för veckodiagrammet (standard 2022).
- `--png` sparar diagrammen som PNG vid sidan av PDF-filerna.
- Enhetstesterna körs med: `python -m unittest -v test_butterfly_analysis`

Programmet skriver ut en sammanfattning per fil, samt vilka rader som inte kunde tolkas. En trasig fil avbryter inte körningen: övriga filer analyseras ändå.

3. Bibliotek och installation

Utöver Pythons standardbibliotek (csv, datetime, argparse, collections, pathlib och unittest) används endast matplotlib. Pandas används inte. Installation:

```
python -m pip install -r requirements.txt
```

Programmet är skrivet för Python 3.9 eller senare (det använder `date.fromisocalendar` och typannoteringar via `from __future__ import annotations`).

4. Så är programmet strukturerat

Koden är uppdelad i tre moduler efter ansvar, plus ett huvudprogram och en testfil. Uppdelningen gör att beräkningarna kan testas utan att några filer eller diagram skapas:

- `observations.py` – klassen `Observation` (en observation, med metoder för att tolka fälten `Artnamn`, `Antal`, `Nord` och `Slutdatum`) och klassen `ObservationReader` som läser en CSV-fil till `Observation`-objekt. Problem returneras som `ReadIssue`-objekt i stället för att skrivas ut, så att inläsningen går att testa.
- `analysis.py` – klassen `SpeciesAnalysis` som gör alla beräkningar för en art: nordligaste observation per år, observationer och individer per år, veckofördelning och 90 %-perioden. Klassen innehåller varken `print`, `open` eller `matplotlib`.
- `plotting.py` – klassen `FigureWriter` som ritar och sparar de tre diagrammen. All `matplotlib`-kod ligger här.
- `butterfly_analysis.py` – huvudprogram. Klassen `ButterflyApplication` sköter användarinteraktionen: väljer filer och art, skriver ut sammanfattning och felmeddelanden och anropar de övriga modulerna. Utanför klasserna finns bara `parse_args` och `main`.
- `test_butterfly_analysis.py` – 39 enhetstester (unittest).
- `butterfly_data/` – datafilerna, `plots/` – de genererade diagrammen.

Reflektioner kring lösningen

Koddesign

Den bärande tanken är att skilja beräkning från användarinteraktion. SpeciesAnalysis returnerar bara värden, FigureWriter ritar dem och ButterflyApplication skriver ut dem. Det gör beräkningarna testbara: ett test kan kontrollera att 90 %-perioden blir vecka 20–21 utan att någon fil skapas. Samma princip gäller inläsningen, där ObservationReader returnerar ReadIssue-objekt i stället för att skriva ut felen själv – huvudprogrammet bestämmer att de tio första visas och resten sammanfattas, vilket behövs eftersom filerna har tiotusentals rader.

Algoritmer

Alla sammanställningar görs med en genomgång av observationslistan. Nordligaste observationen per år är ett löpande maximum per år, och antalet observationer per år en räknare per år. 90 %-perioden räknas ut genom att gå igenom veckorna i ordning och summera antalet observationer: den första veckan där den kumulativa andelen når 5 % blir periodens start och den första där den når 95 % blir dess slut. Alla stegen är linjära i antalet observationer, och en körning över alla sex datafilerna (drygt 130 000 rader) tar några sekunder.

Datastrukturer

Observationerna hålls som en lista av Observation-objekt med `__slots__`, vilket sparar minne när listan innehåller tiotusentals element. Årsvisa resultat returneras som ordböcker med året som nyckel, eftersom åren är glesa – en art behöver inte ha observationer varje år. Veckofördelningen är i stället en lista med 53 platser, eftersom veckonumren är tätt packade och alltid ligger i samma intervall; då blir både uppslagning och den kumulativa summeringen enkel. Observationerna grupperas efter ISO-år, så att dagarna kring nyår hamnar i den vecka de faktiskt tillhör.

Felhantering

Felen delas upp efter hur allvarliga de är. En rad utan tolkbart artnamn eller slutdatum kan inte placeras i något diagram och hoppas över med ett meddelande som anger radnummer och orsak. Saknad nordkoordinat är mindre allvarligt: observationen behålls och räknas för sitt år, men används inte i utbredningsdiagrammet. Enligt uppgiften tolkas "noterad" i kolumnen Antal som en individ. Fel som gäller hela filen – saknad rubrikrad, saknade kolumner eller en fil som inte går att öppna – lyfts som ObservationError och fångas i huvudprogrammet, som rapporterar felet och går vidare till nästa fil. Filer som inte är UTF-8 läses om med CP1252 innan de ges upp. Osäkra bestämningar, som Artportalen skriver "[Amiral]", räknas till samma art i stället för att kastas bort.

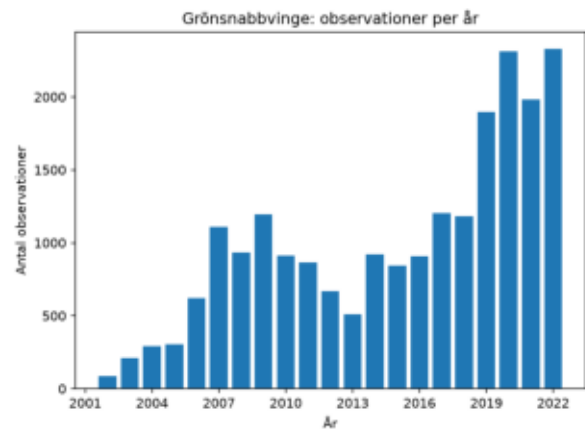
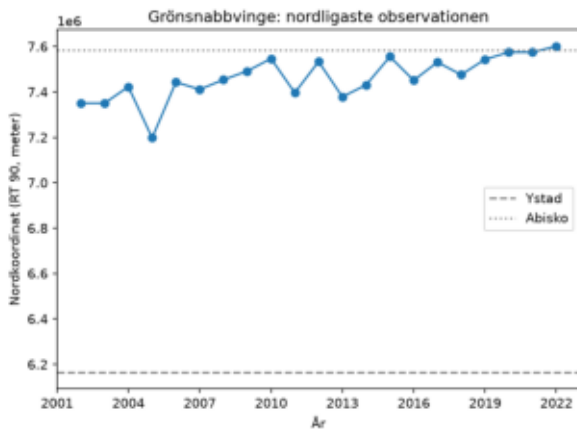
Testning

Programmet har 39 enhetstester i `test_butterfly_analysis.py`, körda med unittest. De täcker tolkningen av enskilda fält (datum, antal, koordinat, artnamn inom hakparenteser), inläsningen av hela filer (fält med semikolon inom citattecken, byte order mark från Excel, filer med fel teckenkodning, saknade kolumner, trasiga rader), beräkningarna (maximum per år, räknare per år, veckofördelning, 90 %-perioden med och utan data) samt att programmet fortsätter med nästa fil när en fil är trasig. Testerna använder temporära kataloger och lämnar inga filer efter sig.

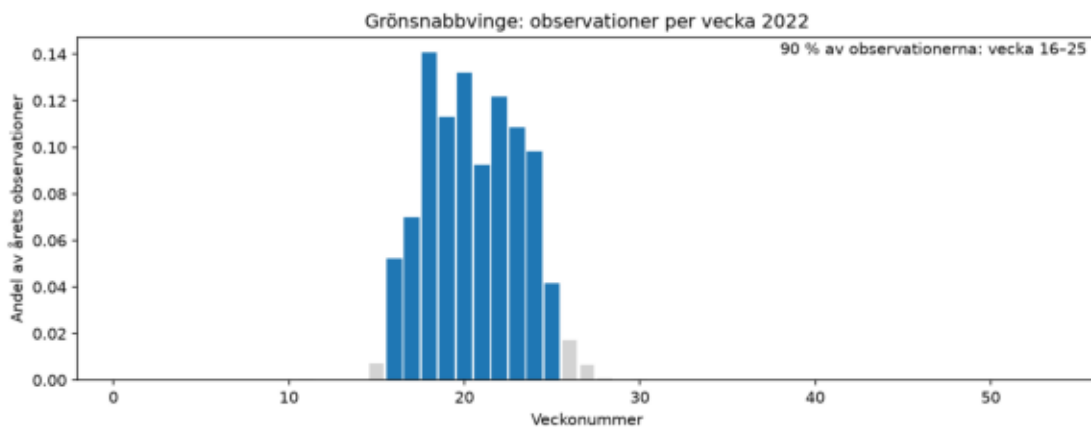
Kontrollräkning mot data: Grönsnabbvinge har 2 330 observationer 2022 och 90 %-perioden vecka 16–25, vilket stämmer med en oberoende uträkning direkt ur CSV-filen.

Appendix: exempelfigurer och biologiska frågor

Figurena nedan är programmets utdata för Grönsnabbvinge (kommandot i punkt 2). Motsvarande figurer för samtliga sex arter finns i katalogen plots/.



Till vänster nordligaste observationen per år, med linjer för Ystad och Abisko; till höger antalet observationer per år.



Andel observationer per vecka 2022. De blå staplarna är perioden då 90 % av observationerna görs.

Svar på frågorna i uppgiften

Nordligaste observationen i början respektive slutet av perioden (mil norrut i RT 90, medelvärde över tre år) och 90 %-perioden 2022:

Amiral: 726 → 751 mil, vecka 21-39

Grönsnabbvinge: 737 → 758 mil, vecka 16-25

Sälgskimmerfjäril: 630 → 685 mil, vecka 26-31

Skogspärlemorfjäril: 711 → 724 mil, vecka 26-32

Sorgmantel: 732 → 757 mil, vecka 12-35

Tistelfjäril: 732 → 748 mil, vecka 21-37

Vilka arter verkar ha flyttat norrut? Samtliga arter har sin nordligaste observation längre norrut i slutet av perioden än i början. Tydligast är Sälgskimmerfjärilen.

Är antalet observationer konstant? Nej, det ökar kraftigt för alla arter. Det beror troligen mer på att Artportalen fått fler användare än på att fjärilarna blivit fler – något att ha i åtanke även när man tolkar utbredningen.

Finns det mönster i när fjärilarna är aktiva? Ja, och de skiljer sig mellan arterna.

Grönsnabbvinge är en vårfjäril med en kort period i maj-juni, medan Amiral och Tistelfjäril, som flyger in söderifrån, syns sent och under betydligt längre tid. Sorgmantel har den längsta perioden eftersom den övervintrar som fullbildad.